## 01 / Source of truth

# Trader Records API

Credosis Internship Program - Phase 02 backend demonstration

Implementation specification v1.0 | 3 November 2026 | Owner: Agboji Simon Agboji

### Purpose

Help a small trader build a persistent record of self-reported business activity. The API accepts structured transactions saved by a future offline client, prevents duplicate writes on retry, computes a cash-flow summary, and allows a specifically authorised lender to view that summary.

### Agreed implementation

One runnable ASP.NET Core .NET 9 project, organised into API, Application, Infrastructure, and Domain folders. Scalar for demonstration; EF Core with SQLite; lightweight Result pattern; explicit dependency injection; EF configuration assembly scanning.

### Status and authority

This document specifies planned behaviour. It is not evidence that features already exist. OpenCode must implement incrementally; Simon reviews and explains each increment. All implementation requirements begin as Planned. Mark Implemented only after code review and Verified only after the stated checks pass.

Changes to routes, DTOs, business rules, package pins, or scope require an updated specification and a brief decision log before dependent work continues. Examples are illustrative; the numbered rules and contracts are authoritative.

### Delivery

Submit an accessible GitHub repository and a 3-5 minute English walkthrough. A public live API is optional under the supplied recruitment email. Deadline: Sunday 8 November 2026, 6:59 p.m. Africa/Lagos (11:59 p.m. UTC+6). Internal target: Saturday 7 November.

# 02 / Scope and functional requirements

| ID | Requirement and acceptance criterion |
|---|---|
| FR-01 | Trader registration creates a Trader user and one business atomically. Duplicate normalised email returns 409; a caller cannot register as Lender. |
| FR-02 | Login verifies a hashed password and returns a 60-minute JWT. Invalid email/password returns the same generic 401 response. |
| FR-03 | Authenticated traders submit 1-100 entries per sync request. Every structurally readable item receives an ordered outcome; rejected items do not prevent valid items being saved. |
| FR-04 | Identical retries return Duplicate and do not change totals. An existing client ID with changed canonical fields returns Rejected / CLIENT_ID_CONFLICT. |
| FR-05 | Trader summary returns exactly eight weekly buckets, totals, and a transparent record consistency indicator. Empty history produces zero totals and zero score. |
| FR-06 | Owner can grant or revoke access for a seeded lender. Repeated grant/revoke requests are idempotent. |
| FR-07 | A Lender can view a business summary only while consent is active. Every request checks current persisted consent. |
| FR-08 | Restarting the API preserves stored data. Development startup applies migrations and seeds demo records without multiplying them. |
| FR-09 | Scalar describes bearer authentication, request/response schemas, role restrictions, statuses, examples, and error contracts. |

### Explicitly excluded from v1

No chat frontend, mobile app, offline browser storage, background client retry mechanism, WhatsApp integration, natural-language parsing, machine learning, loan approval, bank/payment integration, creditworthiness prediction, production deployment, refresh tokens, password recovery, public lender registration, transaction edit/delete, or record verification.

The offline claim is limited to server-side support for safe synchronisation after reconnection. Scalar simulates the future client. Consent controls future API reads; it cannot erase information a lender previously downloaded.

# 03 / Stack and version policy

| Dependency | Pinned baseline / decision |
|---|---|
| Target framework / language | net9.0 / C# 13; nullable reference types and implicit usings enabled. |
| ASP.NET Core runtime | 9.0.21 baseline. Install an appropriate .NET 9 SDK and record its exact installed version in global.json before the first implementation commit. |
| Microsoft.EntityFrameworkCore.Sqlite | 9.0.21 |
| Microsoft.EntityFrameworkCore.Design | 9.0.21; PrivateAssets=all |
| dotnet-ef local tool | 9.0.21; commit .config/dotnet-tools.json |
| Microsoft.AspNetCore.Authentication.JwtBearer | 9.0.21 |
| Microsoft.AspNetCore.OpenApi | 9.0.21 |
| Scalar.AspNetCore | 2.0.30; verified .NET 9 integration baseline, not a claim of latest version. |
| Password hashing | PasswordHasher<User> from ASP.NET Core shared framework; no custom hash algorithm. |
| Test project only | xunit 2.9.3; xunit.runner.visualstudio 3.0.2; Microsoft.NET.Test.Sdk 17.12.0; Microsoft.AspNetCore.Mvc.Testing 9.0.21. |
| Database / mapping | SQLite through the pinned EF provider; no separately installed server; manual DTO mapping. |

These are explicit baseline pins, not a claim that the complete application has been restored or tested. Before feature work, restore, build, audit dependencies, and smoke-test Scalar authentication on Simon's machine. Commit packages.lock.json and use locked restore after the initial successful restore.

If a pin has a security advisory or cannot restore, record the reason and select a compatible patched version before implementation. Keep Microsoft 9.0.x packages and dotnet-ef aligned. Do not silently switch to .NET 10 or use floating package versions.

SDK patch is intentionally a setup gate: the environment used to author this specification did not establish Simon's installed SDK. OpenCode must report dotnet --info, select an installed .NET 9 SDK, and insert its exact version into global.json. Do not confuse an SDK version such as 9.0.xxx with runtime 9.0.21.

The xUnit v2 baseline is retained for a small familiar test setup; its package page directs new feature development to v3. No migration is required for this recruitment demo. SQLite native library version is transitive: capture the resolved dependency graph rather than invent a separate engine pin.

# 04 / Single-project organisation

| Folder / file | Responsibility |
|---|---|
| API/Controllers | AuthController, TransactionsController, BusinessesController, ConsentsController, LenderBusinessesController. |
| API/ErrorHandling | GlobalExceptionHandler; shared Result-to-ProblemDetails mapping and authentication failure formatting. |
| Application/Common | Result<T>, Error, ErrorType, Unit; no HTTP status codes inside service results. |
| Application/DTOs | Auth, Sync, Summary and Consent request/response contracts. |
| Application/Interfaces | IAuthService, ITransactionSyncService, ISummaryService, IConsentService, ITokenService. |
| Application/Services | AuthService, TransactionSyncService, SummaryService, ConsentService. |
| Domain/Entities and Domain/Enums | User, Business, Transaction, Consent; UserRole and TransactionType. |
| Infrastructure/Data | AppDbContext, Configurations/*Configuration, Migrations, DevelopmentSeeder. |
| Infrastructure/Authentication | JwtOptions, JwtTokenService. |
| Program.cs and root files | Composition root, middleware, development startup; appsettings, csproj, global.json, README, package lock. |
| Sibling TraderRecords.Tests | Automated tests; references the single API project. It is not another deployed application. |

The solution may contain one API project and one test project. No separate Domain/Application/Infrastructure assemblies. The folders organise code; strict Clean Architecture dependency inversion is not a goal. Application services may directly inject AppDbContext. Domain entities remain simple and have no controller dependencies.

No generic repository, extra UnitOfWork wrapper, MediatR, CQRS, AutoMapper, event bus, cache, or service-scanning library. Controllers do not query EF directly. Infrastructure contains persistence and token creation, not transaction business rules.

## Code readability rule

OpenCode must add useful comments explaining intent and non-obvious behaviour, especially money conversion, duplicate comparison, concurrency recovery, consent checks and time windows. Avoid comments that merely repeat a statement. Every stage must include a plain-language explanation Simon can defend.

# 05 / Database conventions and relationships

C# entity IDs are Guid values generated by the server. ClientId is a non-empty Guid generated by the future client and reused across retries. Guid.Empty is invalid for incoming identifiers. JSON uses camelCase properties and enum names as strings.

Money is stored and exposed as signed 64-bit integer kobo (AmountMinor). Currency is fixed to NGN. Example: NGN 45,000.00 = 4,500,000 kobo. Individual transaction range: 1 through 100,000,000,000 kobo inclusive. Do not use float/double for money. Use checked summation and map numeric overflow to a controlled server error.

All persisted timestamps are UTC DateTime mapped consistently to SQLite TEXT. HTTP occurrence timestamps must include Z or an explicit offset; parse as DateTimeOffset then normalise to UTC. Use TimeProvider for the current instant. Do not compare SQLite DateTimeOffset values for range queries.

| Relationship | Database rule |
| --- | --- |
| User -> Business | One Trader owns at most one business. Unique Business.OwnerUserId. FK to Users.Id; delete Restrict. |
| Business -> Transaction | One business has many transactions. FK Transaction.BusinessId; delete Restrict. |
| Business -> Consent | One business has many lender consents. FK Consent.BusinessId; delete Restrict. |
| User -> Consent | Consent.LenderUserId references Users.Id; delete Restrict. Service verifies target user is Lender. |
| Idempotency | Unique index on Transactions(BusinessId, ClientId). |
| Consent identity | Unique index on Consents(BusinessId, LenderUserId). |
| Summary queries | Non-unique index on Transactions(BusinessId, OccurredAtUtc). |
| User lookup | Unique index on Users.NormalizedEmail. |

Use NOT NULL for required columns and CHECK constraints for enum values, positive bounded AmountMinor, NGN currency, and bounded string lengths. SQLite does not enforce HasMaxLength alone: add explicit length checks for fields where a database length guarantee is specified. No public deletion endpoints in v1.

# 06 / Entity models: users and businesses

| User property | Type / nullability | Rule |
|---|---|---|
| Id | Guid / required | Primary key. |
| FullName | string / required | Trimmed; 2-100 characters. |
| Email | string / required | Trimmed; valid email shape; max 254. |
| NormalizedEmail | string / required | Email.ToUpperInvariant(); unique. |
| PasswordHash | string / required | Framework-generated hash; never returned or logged. |
| Role | UserRole / required | Trader=1, Lender=2; persisted string with CHECK. |
| CreatedAtUtc | DateTime / required | Server UTC timestamp. |

| Business property | Type / nullability | Rule |
|---|---|---|
| Id | Guid / required | Primary key. |
| OwnerUserId | Guid / required | Unique FK to User. |
| Name | string / required | Trimmed; 2-120 characters. |
| Currency | string / required | Exactly NGN; immutable in v1. |
| CreatedAtUtc | DateTime / required | Server UTC timestamp. |

## Registration boundaries

RegisterRequest contains fullName, email, password, businessName only. FullName and businessName use the bounds above. Password length is 12-128 characters; never trim it. No role or owner ID accepted. Hash with IPasswordHasher<User>; create user and business in one database transaction. On normalised-email uniqueness races, return EMAIL_IN_USE (409).

Only development seeding creates lender accounts. This prevents public callers from elevating themselves to lender status. Public trader registration is included to demonstrate creation; the video can use seeded users for speed.

## Navigations

User.Business is optional because Lenders own no business. Business.Owner is required. Business.Transactions and Business.Consents are collections. Consent.Lender is required. Return DTOs rather than serialising EF entities or navigation graphs.

# 07 / Entity models: transactions and consent

| Transaction property | Type / nullability | Rule |
|---|---|---|
| Id | Guid / required | Server-generated primary key. |
| BusinessId | Guid / required | From authenticated trader ownership. |
| ClientId | Guid / required | Non-empty client ID; unique within business. |
| Type | TransactionType / required | Sale=1 or Expense=2; persisted string with CHECK. |
| AmountMinor | long / required | 1..100000000000 kobo. |
| Note | string / optional | Trimmed; empty becomes null; max 280. |
| OccurredAtUtc | DateTime / required | Canonical client occurrence instant. |
| ReceivedAtUtc | DateTime / required | Server receipt instant; never client supplied. |

| Consent property | Type / nullability | Rule |
|---|---|---|
| Id | Guid / required | Server primary key. |
| BusinessId | Guid / required | Owned by acting Trader. |
| LenderUserId | Guid / required | Must identify a Lender. |
| GrantedAtUtc | DateTime / required | Most recent activation time. |
| RevokedAtUtc | DateTime? / optional | Null = active; otherwise revoked. |

Consent is an upserted current-state row, not a complete audit history. Repeat grant while active leaves timestamps unchanged. Regrant after revocation updates GrantedAtUtc and clears RevokedAtUtc. Repeat revoke leaves the original revocation timestamp unchanged. No expiry in v1.

Transactions are immutable. A correction workflow is excluded. Canonical duplicate comparison uses Type, AmountMinor, normalised Note, and OccurredAtUtc. ReceivedAtUtc and server Id are never part of that comparison. Enforce millisecond timestamp precision during normalisation for both insert and comparison.

Example: the same ClientId with 4,500,000 kobo twice is Duplicate if all canonical fields match. The same ClientId with 5,000,000 kobo is CLIENT_ID_CONFLICT. Neither case creates or updates a row.

# 08 / DTOs and Result contracts

| Contract | Fields / meaning |
|---|---|
| RegisterRequest | fullName:string; email:string; password:string; businessName:string. |
| LoginRequest | email:string; password:string. |
| AuthResponse | userId:Guid; businessId:Guid?; role:string; accessToken:string; tokenType="Bearer"; expiresAtUtc:UTC timestamp. |
| SyncBatchRequest | items:array of SyncItemRequest; 1-100 entries. |
| SyncItemRequest | clientId:Guid?; type:string?; amountMinor:long?; note:string?; occurredAt:DateTimeOffset?. Nullable fields permit per-item missing-field validation. |
| SyncBatchResponse | receivedCount:int; acceptedCount:int; duplicateCount:int; rejectedCount:int; results:ordered SyncItemResult[]. |
| SyncItemResult | index:int (zero-based); clientId:Guid?; status:Accepted\|Duplicate\|Rejected; transactionId:Guid?; errors:ItemError[]. |
| ItemError | code:string; field:string?; message:string. Accepted/Duplicate have an empty error list. |
| ConsentResponse | businessId:Guid; lenderId:Guid; isActive:bool; grantedAtUtc:timestamp; revokedAtUtc:timestamp?. |
| Result<T> | IsSuccess:bool; Value:T only on success; Error:Error only on failure. Enforce invariant using factory methods. |
| Error / ErrorType | Error has Code, Message, Type. Type is Validation, Unauthorized, Forbidden, NotFound, Conflict. Unit represents successful operations with no payload. |

Do not apply nested automatic validation attributes that reject an entire sync batch for one semantic item error. Envelope validation is request-wide; readable item validation belongs in the sync service. A JSON number that cannot bind to Int64 or an invalid Guid/date string may cause request-wide 400 because deserialisation cannot construct the DTO. Document this boundary.

Expected failures return Result<T>; unexpected faults throw and are centrally handled. Business services return no IActionResult and no HTTP-specific types. Do not wrap successful responses in a redundant generic success envelope.

# 09 / Service interfaces and responsibilities

All asynchronous methods take CancellationToken ct as their last argument. userId and lenderId used for the acting identity are extracted by the controller from validated claims, never from request body ownership fields.

| Interface / operation | Signature contract and responsibility |
|---|---|
| IAuthService.RegisterAsync | (RegisterRequest request, ct) -> Task<Result<AuthResponse>>. Validate, normalise email, hash password, atomically create Trader/business, issue token. |
| IAuthService.LoginAsync | (LoginRequest request, ct) -> Task<Result<AuthResponse>>. Verify hash, optionally rehash when needed, issue token. Generic failure message. |
| ITransactionSyncService.SyncAsync | (Guid traderUserId, SyncBatchRequest request, ct) -> Task<Result<SyncBatchResponse>>. Resolve business, validate envelope/items, persist safely, preserve input order. |
| ISummaryService.GetForTraderAsync | (Guid traderUserId, ct) -> Task<Result<BusinessSummaryResponse>>. Resolve owned business; compute standard window. |
| ISummaryService.GetForLenderAsync | (Guid lenderUserId, Guid businessId, ct) -> Task<Result<BusinessSummaryResponse>>. Check business existence and active consent, then compute same summary. |
| IConsentService.GrantAsync | (Guid traderUserId, Guid lenderId, ct) -> Task<Result<ConsentResponse>>. Validate lender and upsert activation. |
| IConsentService.RevokeAsync | (Guid traderUserId, Guid lenderId, ct) -> Task<Result<Unit>>. Validate lender; revoke row if present; absent consent succeeds. |
| ITokenService.Create | (User user, Guid? businessId) -> AuthResponse. Sign JWT using validated options and current TimeProvider. |

SummaryService shares a private computation path after access checks. ConsentService never accepts a business ID from a trader request. AuthService alone handles password verification. JwtTokenService alone knows signing details. Simple validation helpers may be private methods; do not add a new abstraction for every method.

Controllers perform binding, authorisation attributes, identity extraction, service calls, and Result-to-HTTP translation. Services enforce business ownership and consent even when the caller supplies valid-looking IDs.

# 10 / API contract and controller map

| Method / route | Controller / access | Success and expected errors |
|---|---|---|
| POST /api/auth/register | Auth / Anonymous | 201 AuthResponse; 400 invalid; 409 email in use. No Location header required because no public user-resource GET exists. |
| POST /api/auth/login | Auth / Anonymous | 200 AuthResponse; 400 invalid payload; 401 incorrect credentials. |
| POST /api/transactions/sync | Transactions / Trader | 200 SyncBatchResponse, including all-item rejection; 400 invalid envelope; 401; 403 wrong role; 404 owned business missing. |
| GET /api/businesses/me/summary | Businesses / Trader | 200 BusinessSummaryResponse; 401; 403; 404 business missing. |
| PUT /api/businesses/me/consents/{lenderId:guid} | Consents / Trader | 200 ConsentResponse; 400 empty ID; 401; 403; 404 unknown lender/business; 409 target is not Lender. |
| DELETE /api/businesses/me/consents/{lenderId:guid} | Consents / Trader | 204 no body, including already absent/revoked; 400 empty ID; 401; 403; 404 unknown lender/business; 409 target is not Lender. |
| GET /api/lender/businesses/{businessId:guid}/summary | LenderBusinesses / Lender | 200 BusinessSummaryResponse; 400 empty ID; 401; 403 no consent/wrong role; 404 unknown business. |

Use [ApiController] and attribute routes. GUID route constraints yield routing 404 for malformed route segments. Empty but syntactically valid GUIDs yield validation 400. Cross-trader access is impossible through /me ownership resolution; no trader route accepts another owner ID.

All endpoints use application/json except ProblemDetails (application/problem+json) and empty 204 responses. Camel-case JSON. Do not expose password hashes, tokens in logs, raw database exceptions, or entity navigation properties.

## OpenAPI requirements

Add an HTTP bearer JWT security scheme and operation security requirements for protected routes. Scalar must allow a token entered by the reviewer. Describe role restrictions explicitly; a security scheme alone does not enforce authorisation. Map /openapi/v1.json and /scalar in Development.

# 11 / Sync rules and persistence algorithm

Capture one server now per batch. Envelope is required, items cannot be null, count must be 1-100, and null array entries are rejected per item with INVALID_ITEM. Restrict request body to 256 KiB; oversized transport requests return 413 before processing.

For each readable item: non-empty ClientId; Type exactly Sale or Expense; bounded positive integer AmountMinor; Note <=280 after trimming; occurrence timestamp required with offset; normalise UTC to milliseconds. New entries must fall between now minus 365 days and now inclusive. No future tolerance.

Order of decisions: validate canonical field shape first; look up the scoped ClientId; if present compare canonical values and return Duplicate or CLIENT_ID_CONFLICT; if absent apply the new-entry age window and insert. This allows a previously accepted historical entry to remain a duplicate when retried after the 365-day admission window.

## Small, reliable implementation

Process items sequentially; never use parallel EF operations on the scoped DbContext. Save each valid new item separately to support partial success. A repeated ID later in the same batch sees the earlier insert. One failed item must not roll back unrelated accepted items. This is intentionally not an all-or-nothing batch.

The unique database index is the final guard against concurrent requests. If an insert fails specifically for the scoped client-ID unique constraint, detach the failed Added entity, re-read the stored row, compare canonical values, and classify it. Never classify every DbUpdateException as a duplicate.

For SQLite busy/locked transient errors, use a short bounded retry (maximum three attempts); if exhausted return request-wide 503 with ProblemDetails. Other unexpected failures return 500. Some earlier entries may already be saved when either happens; the client safely retries the full batch. Do not return a success response before its Accepted entries are persisted.

| Item error code | Meaning |
| --- | --- |
| CLIENT_ID_REQUIRED / INVALID_ITEM | Missing/empty client ID or null item. |
| INVALID_TYPE / INVALID_AMOUNT | Type not recognised or amount outside limits. |
| NOTE_TOO_LONG / INVALID_OCCURRED_AT | Oversized note, missing time, future time or new entry too old. |
| CLIENT_ID_CONFLICT | Existing ID reused with different canonical content. |

Response counts must partition receivedCount. Results remain in input order; transactionId exists only for Accepted/Duplicate. Duplicate is a successful retry outcome, not HTTP 409. Conflicts inside a batch remain item-level rejection under HTTP 200.

# 12 / Sync examples and expected outcomes

## Request example in field form

| Index | Fields |
|---|---|
| 0 | clientId = 11111111-1111-4111-8111-111111111111; type = Sale; amountMinor = 4500000; note = Sold rice; occurredAt = 2026-11-03T09:00:00+01:00 |
| 1 | clientId = 22222222-2222-4222-8222-222222222222; type = Expense; amountMinor = 1200000; note = Stock purchase; occurredAt = 2026-11-03T09:05:00+01:00 |
| 2 | clientId = 33333333-3333-4333-8333-333333333333; type = Sale; amountMinor = -100; note = null; occurredAt = 2026-11-03T09:10:00+01:00 |

The request envelope is an object with an items array containing these three objects. These dates assume the demonstration clock is later than their occurrence times. README examples should use current past timestamps or explicitly document the reference clock.

## First request

HTTP 200. receivedCount=3, acceptedCount=2, duplicateCount=0, rejectedCount=1. Results at indexes 0 and 1 are Accepted with server transaction IDs; index 2 is Rejected with INVALID_AMOUNT. Totals increase by 4,500,000 sales kobo and 1,200,000 expense kobo.

## Identical retry

HTTP 200. receivedCount=3, acceptedCount=0, duplicateCount=2, rejectedCount=1. Accepted IDs from the first call now appear as Duplicate using the same transaction IDs. The invalid entry remains rejected. Totals do not change.

## Changed retry

Resend index 0 with the same ClientId and amountMinor=5000000. The item is Rejected / CLIENT_ID_CONFLICT. The original 4,500,000-kobo sale remains unchanged. This response contains an item error; it does not return request-wide HTTP 409.

## Offline demonstration boundary

Scalar sends structured data when the backend is reachable. It does not store phone entries offline, parse a sentence, or schedule background synchronisation. Say: "This endpoint supports a future offline client by accepting retry-safe batches." Do not claim the video demonstrates an implemented offline phone app.

# 13 / Summary model and consistency indicator

Use a fixed eight-week reporting window: current Africa/Lagos calendar week plus the preceding seven weeks. Weeks start Monday at 00:00 Lagos time. Convert boundaries to UTC for database filters. Window is [start, next Monday boundary), but include only occurrences <= captured now. Obtain Lagos time via TimeZoneInfo (IANA Africa/Lagos; Windows fallback W. Central Africa Standard Time).

| BusinessSummaryResponse | Definition |
|---|---|
| businessId, businessName, currency | Identifiers/name and fixed NGN. |
| generatedAtUtc, windowStartUtc, windowEndExclusiveUtc | Server capture time and eight-week bounds. |
| totalSalesMinor, totalExpensesMinor | Sum of Sale and Expense amounts in the window. |
| netCashFlowMinor, transactionCount | Sales minus expenses; count of all included transactions. |
| weekly: WeeklySummary[8] | Chronological buckets, including weeks with no data. |
| indicator: ConsistencyIndicator | score:int; activeDays:int; activeWeeks:int; historySpanWeeks:int; disclaimer:string; guidance:string[]. |

WeeklySummary fields: weekStartLocalDate (yyyy-MM-dd), salesMinor, expensesMinor, netCashFlowMinor, transactionCount, recordedDays. A recorded day is a distinct Lagos occurrence date with at least one accepted transaction of either type. Count by occurrence date, not receipt date.

### Formula: record-consistency-v1

activeDays = distinct recorded days in the window. activeWeeks = buckets with at least one transaction. historySpanWeeks = 0 if empty; otherwise number of weekly buckets from the earliest occupied bucket through the current bucket, inclusive (1-8).

score = round-away-from-zero(50 x activeDays / 56 + 30 x activeWeeks / 8 + 20 x historySpanWeeks / 8). Clamp 0-100. All calculations are deterministic given records and now. No stored Scores table. No profit margin component.

Example: 12 active days, 4 active weeks, historySpanWeeks=4 gives round(10.714 + 15 + 10)=36. Empty history gives 0. Every day across eight weeks gives 100. The current partial week makes 100 harder to reach; this fixed-window limitation is disclosed.

Always return disclaimer: "Based on self-reported records; not verified income or a creditworthiness assessment." Guidance: no data -> start recording; activeWeeks<8 -> build weekly history; activeDays<40 -> record activity consistently. This measures recording coverage and span, not truthfulness, turnover quality, or repayment ability. Net cash flow is not accounting profit.

# 14 / Authentication, consent and error handling

JWT: HS256, 60-minute expiry; claims include sub=user ID, role, jti and optional businessId. Configure explicit NameClaimType and RoleClaimType; validate issuer, audience, lifetime and signing key; clock skew 30 seconds. Ownership queries use sub; a businessId claim is informational and is not the authority for ownership.

JwtOptions: Issuer=TraderRecords.Api, Audience=TraderRecords.Demo, ExpiryMinutes=60, SigningKey from user-secrets or environment (Jwt__SigningKey), minimum 32 random bytes. Validate options at startup. No secrets in Git or PDF. Do not prefill real bearer tokens in Scalar.

Authorisation roles: Trader for sync/owner summary/consents; Lender for lender summary. Anonymous protected calls return 401; wrong role 403. Each lender summary checks consent in the database. A request already authorised before revocation may complete; later checks must deny access. Consent covers summary only, not raw transaction notes or credentials.

| Error category | HTTP / code examples |
|---|---|
| Validation | 400 VALIDATION_FAILED; field errors when applicable. |
| Authentication | 401 INVALID_CREDENTIALS or UNAUTHORIZED; generic messages. |
| Authorisation | 403 FORBIDDEN or CONSENT_REQUIRED. |
| Missing resource | 404 BUSINESS_NOT_FOUND or LENDER_NOT_FOUND. |
| Conflict | 409 EMAIL_IN_USE or TARGET_NOT_LENDER. |
| Transport limit / transient store issue | 413 PAYLOAD_TOO_LARGE; 503 DATABASE_BUSY. |
| Unexpected exception | 500 INTERNAL_ERROR; generic detail and server-side correlation log. |

ProblemDetails fields: type (stable URI such as urn:traderrecords:error:validation), title, status, detail, instance=request path; extensions code and traceId. Validation adds errors keyed by field. Example missing consent: title=Forbidden, status=403, code=CONSENT_REQUIRED, detail="Active consent is required."

Use AddProblemDetails, AddExceptionHandler<GlobalExceptionHandler>, UseExceptionHandler and customised invalid-model responses. Configure JWT challenge/forbidden responses to the same format, retaining WWW-Authenticate on 401. Routing/status-code middleware should format otherwise-empty API 404 responses. Never include stack traces in client responses. A body limit enforced before application middleware may emit a platform 413; document and test the local server behaviour.

# 15 / DbContext, configuration and DI

## AppDbContext

Constructor receives DbContextOptions<AppDbContext>. Expose DbSet<User> Users, DbSet<Business> Businesses, DbSet<Transaction> Transactions and DbSet<Consent> Consents. OnModelCreating calls base.OnModelCreating, then ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly). All four IEntityTypeConfiguration<T> classes live in that same API assembly.

Configuration scanning applies mappings only. It does not discover service registrations, run migrations, or seed accounts. Specify ToTable names, keys, foreign keys, unique indexes, string enum conversions, UTC conversion convention, NOT NULL and CHECK constraints explicitly. Read-only summaries use AsNoTracking and filtered queries.

## Database settings

ConnectionStrings:DefaultConnection = Data Source=App_Data/traderrecords.db;Foreign Keys=True;Default Timeout=5. Resolve the path against ContentRootPath so starting from another working directory does not select a different database. Create App_Data when missing. Keep writable database files outside wwwroot.

Use migrations, never EnsureCreated together with migrations. Development startup creates an IServiceScope, resolves the DbContext, calls Database.MigrateAsync, then the idempotent DevelopmentSeeder. Non-development startup does not silently migrate or seed. Tests use an isolated database and explicit setup.

| Registration | Lifetime / reason |
|---|---|
| AppDbContext with UseSqlite | Scoped; one request unit of EF tracking; not thread-safe. |
| IAuthService -> AuthService | Scoped; depends on the scoped context. |
| ITransactionSyncService -> TransactionSyncService | Scoped; per-request persistence. |
| ISummaryService -> SummaryService | Scoped; per-request access checks and reads. |
| IConsentService -> ConsentService | Scoped; consent persistence. |
| ITokenService -> JwtTokenService | Scoped; simple consistent service lifetime. |
| IPasswordHasher<User> -> PasswordHasher<User> | Scoped; framework password hashing. |
| TimeProvider.System | Singleton; replace with a fixed provider in tests. |
| JwtOptions / DevelopmentSeeder | Validated options; seeder scoped, Development only. |

Program.cs explicitly registers controllers, JSON settings, OpenAPI, bearer authentication, role authorisation, problem handling and services. Middleware sequence: exception handler, API empty-status handling, authentication, authorisation, controller endpoints. Add HTTPS redirection when using a trusted local HTTPS profile; provide a loopback-only HTTP profile for reviewers if needed.

# 16 / Local setup, seed data and submission

## Reviewer setup sequence

README must state: install the pinned .NET 9 SDK; clone repository; restore local tools and packages; set Jwt:SigningKey using dotnet user-secrets or Jwt__SigningKey; run the API in Development; open the actual Scalar address printed/documented for the chosen launch profile. Provide exact commented commands after the project path and SDK are fixed.

A fresh Development run automatically applies committed migrations and seeds demo data. Stop the process to reset, delete App_Data/traderrecords.db and any -wal/-shm companions, then restart. Reset instructions must explicitly say that they erase local demo records. No public reset endpoint.

## Seed contract

Create deterministic GUID identities for two Trader accounts, their two businesses, and one Lender. Use example.test email addresses (trader1@example.test, trader2@example.test, lender@example.test). Document fictional demo passwords in README, hash them at seed time, and clearly prohibit reuse for real accounts. These passwords are demo credentials, not JWT secrets.

Seed transactions with deterministic ClientIds and dates relative to the current Lagos week across the last eight weeks, without future occurrences. Insert only missing seed identities/entries; do not regenerate history every restart. Existing records and granted consent are not overwritten. Default seed has no active consent. Reset before recording for a reproducible video.

## Repository deliverables

Include source, csproj, global.json, lock file, local tool manifest, migrations, fictional seed logic, tests, README, this PDF in docs/, an implementation-status checklist and exported OpenAPI JSON. Ignore .db files and companions, bin/obj, local secrets, tokens and real financial data. No Docker, hosting or database server required.

Export the actual running /openapi/v1.json to docs/openapi.json after implementation. It must match the final controllers and DTOs. A static specification is inspectable documentation; it does not execute requests without a running API.

## What to submit

GitHub URL and accessible 3-5 minute walkthrough video URL. If the form has an optional live link, leave it blank or explain "Runs locally; setup and API specification are in README." A localhost URL is not publicly accessible. Check repository and video links in a signed-out/private window. If the form unexpectedly requires hosting, reassess from the actual form; do not invent a public URL.

# 17 / Verification matrix and walkthrough

| ID / requirement | Evidence required before Verified |
|---|---|
| T01 / FR-01,02 | Register creates one user/business; duplicate email and simultaneous duplicate registration yield controlled 409; public role escalation impossible; login correct/incorrect. |
| T02 / FR-03,04 | First batch accepts; identical retry does not add rows; changed payload conflicts; duplicate within same batch; mixed valid/invalid; counts/order correct. |
| T03 / FR-03 | 0/101 entries rejected request-wide; amount boundaries; null item; unknown type; future/old timestamp; malformed JSON; body size. |
| T04 / FR-04 | Two requests with same new client ID leave one row, return correct retry outcomes, and never leak a raw unique exception. |
| T05 / FR-05 | Empty data; eight buckets; Lagos Monday boundary; offset normalisation; out-of-window exclusion; exact totals; indicator example=36. |
| T06 / FR-06,07 | Lender denied before grant, allowed after grant, denied after revoke; repeat grant/revoke; wrong lender cannot use another lender's consent. |
| T07 / security | Missing/expired token; wrong role; second Trader cannot read/write first Trader's business; business claim does not override database ownership. |
| T08 / FR-08 | Fresh database migrates/seeds; second startup does not duplicate; restart preserves accepted records; reset restores clean demo. |
| T09 / errors/docs | ProblemDetails contracts; generic 500; Scalar bearer flow; OpenAPI export; signed-out submission link checks. |

Use real SQLite for persistence tests (temporary files or an open in-memory SQLite connection); do not substitute EF InMemory for relational uniqueness tests. Concurrent-write tests use separate contexts/connections and preferably a temporary file. Override TimeProvider for deterministic time-window assertions. API integration tests use WebApplicationFactory with isolated database and test signing settings; expose Program as partial if needed.

### 3-5 minute demonstration

0:00-0:30 explain trader record gap and self-report limitation. 0:30-1:00 log in as trader. 1:00-2:00 sync valid/invalid entries, retry, then show changed-ID conflict. 2:00-2:40 show cash-flow summary and explain indicator. 2:40-3:40 lender denied, trader grants, lender reads, trader revokes, lender denied again. 3:40-4:30 explain unique index, partial success, local setup and one real development challenge.

# 18 / OpenCode implementation gates

Give OpenCode this PDF and implement one gate at a time. Do not ask it to build all features in a single unattended pass. Each gate ends with changed-file list, design explanation, commands run, results, and remaining limitations. Simon reviews before starting the next gate.

| Gate | Work and completion check |
|---|---|
| 1 - skeleton | Establish SDK/pins, single-project folders, controllers and Scalar. Restore/build; inspect actual package graph; create global.json and lock file. |
| 2 - persistence | Entities, mappings, assembly scan, DbContext, migration, development seed. Inspect migration SQL/constraints; start fresh twice; verify persistence. |
| 3 - authentication | Result types, hashing, JWT, register/login, error mapping. Verify roles, claims, generic failures and email race behaviour. |
| 4 - core sync | Implement DTO boundary, item validation, canonical comparison, unique constraint recovery, partial persistence. Run T02-T04 before adding features. |
| 5 - summary | Implement eight Lagos weeks, integer money totals and pure indicator calculation. Verify fixed-clock boundary and formula tests. |
| 6 - consent | Implement grant/revoke and lender read path. Verify before/after grant/revoke and wrong-user access. |
| 7 - delivery | Finish integration tests, README, OpenAPI export, status log, video, clean-clone run and accessible submission links. |

## Initial instruction to OpenCode

"Read the source-of-truth PDF first. Implement Gate 1 only, keeping one runnable API project with API/Application/Infrastructure/Domain folders. Do not add excluded architecture or features. Report the exact .NET 9 SDK and dependency versions, explain every created file, and show restore/build and Scalar smoke-test results. Add meaningful comments. Stop after this gate for my review."

## Implementation register

Track FR-01 through FR-09 as Planned / Implemented / Verified in docs/implementation-status.md. For every Verified item record commit, verification command or manual evidence, date, and remaining limitations. Do not mark a requirement verified merely because generated code compiles.

## Questions Simon must be able to answer

Why client-generated IDs and a database unique index? Why can a batch return 200 with rejected entries? What happens if the connection fails after some inserts? Where is owner identity obtained? What does the indicator measure? Why integer kobo? How is consent checked after revocation? What offline behaviour is still unimplemented?

# 19 / Decisions, constraints and source references

| Decision | Reason / practical limit |
|---|---|
| Single application project | Enough organisation for a small demo without multiple deployment units. |
| SQLite and integer kobo | Portable persistent storage with exact money representation; not a claim of distributed production scalability. |
| Result + central exceptions | Expected failures remain explicit; unexpected faults produce consistent safe errors. |
| Sequential partial sync | Simple per-entry outcomes and safe retries; earlier successes can survive a request-level failure. |
| Summary-only consent | Least data needed for the demonstration; no complete consent audit history in v1. |
| Record consistency indicator | Transparent toy measure of record coverage; cannot verify trading activity or justify lending decisions. |

## Official references - checked 3 November 2026

ASP.NET Core 9.0.21 release
https://github.com/dotnet/aspnetcore/releases/tag/v9.0.21

EF Core SQLite 9.0.21
https://www.nuget.org/packages/Microsoft.EntityFrameworkCore.Sqlite/9.0.21

EF Core Design 9.0.21
https://www.nuget.org/packages/Microsoft.EntityFrameworkCore.Design/9.0.21

JWT bearer 9.0.21
https://www.nuget.org/packages/Microsoft.AspNetCore.Authentication.JwtBearer/9.0.21

OpenAPI 9.0.21
https://www.nuget.org/packages/Microsoft.AspNetCore.OpenApi/9.0.21

Scalar 2.0.30 and .NET 9 integration
https://www.nuget.org/packages/Scalar.AspNetCore/2.0.30

SQLite provider limitations
https://learn.microsoft.com/en-us/ef/core/providers/sqlite/limitations

xUnit baseline
https://www.nuget.org/packages/xunit/2.9.3

Test runner / SDK
https://www.nuget.org/packages/xunit.runner.visualstudio/3.0.2

Microsoft test SDK
https://www.nuget.org/packages/Microsoft.NET.Test.Sdk/17.12.0

Source pages establish package availability and documented integration; only the implementation restore/build/test gates establish the final project works. Project-specific requirements, formulas and contracts are design decisions in this specification. Recruitment requirements are taken from the email supplied by Simon.